Module 5 — Assembly Language Machine Problems

NASM 32-bit | Procedures | Conditional Selection | User Input | ASCII Art

Two interactive console applications • Intermediate difficulty • Recommended time: 2–3 hours per problem

General Programming Requirements

Develop both applications using NASM x86 32-bit assembly on Linux with int 0x80. Programs must compile and run without errors.

Requirement	Minimum
Procedures using CALL and RET	5
User input operations	3
Conditional selections	5
Menu options	4
ASCII art designs	3
Menu repetition or loop	1
Input validation and exit option	Required

MACHINE PROBLEM 1 - LEYNES, ETHAN ANDREI O.

MACHINE PROBLEM 2 - YULO, FRANCIS KARL G.

SECTION : TN25

COURSE CODE: CS0003L COMPUTER SYSTEMS & ARCHITECTURE LAB

Machine Problem 1 — Zodiac Horoscope and Fortune Teller

Problem Statement

Create an interactive Zodiac Horoscope and Fortune Teller System. The user enters a name, birth month and birth day. The program identifies the zodiac sign and displays a fictional horoscope according to a selected category. Separate major tasks into procedures and use conditional branches for date and menu decisions.

1. Welcome Screen

```
========================================
       THE MYSTIC ZODIAC ORACLE
========================================
              *       *
           *     /\      *
                /  \
          *    /____\    *
               |  |
```

```
          *   |__|   *
       DISCOVER YOUR DESTINY!
========================================
```

2. User Information

```
Enter your name: Maria
Enter birth month (1-12): 8
Enter birth day (1-31): 15

Welcome, Maria!
```

Validate month and day against actual month lengths. February 29 may be rejected because no birth year is collected.

3. Zodiac Identification

Sign	Date range
Capricorn	Dec 22–Jan 19
Aquarius	Jan 20–Feb 18
Pisces	Feb 19–Mar 20
Aries	Mar 21–Apr 19
Taurus	Apr 20–May 20
Gemini	May 21–Jun 20
Cancer	Jun 21–Jul 22
Leo	Jul 23–Aug 22
Virgo	Aug 23–Sep 22
Libra	Sep 23–Oct 22
Scorpio	Oct 23–Nov 21
Sagittarius	Nov 22–Dec 21

Display the sign, element, symbol, personality description, and simplified ASCII art corresponding to the selected sign.

```
========================================
         YOUR ZODIAC SIGN
========================================
              /\_/\
             ( o.o )
              > ^ <
Zodiac Sign: LEO
Element: Fire
Symbol: Lion
Personality: Confident, creative, courageous.
```

4. Horoscope Menu

```
[1] Love and Relationships
[2] Career and Studies
[3] Health and Wellness
[4] Lucky Number and Color
[5] View Zodiac Information
[6] Exit
Enter your choice:
```

Choices 1–3 display category-specific fictional predictions; choice 4 displays a lucky number and color; choice 5 shows zodiac details and art; choice 6 exits. Invalid selections must display an error and return to the menu.

5. Sample Interaction

```
Enter your name: Maria
Enter birth month: 8
Enter birth day: 15
Hello, Maria!
Your Zodiac Sign: LEO
Your Element: FIRE
Choice: 2
CAREER HOROSCOPE
Leo, your creativity will help you
solve a difficult problem today.
Stay focused and confident!
Return to menu? (Y/N): Y
```

Required Procedures

Procedure	Responsibility
display_welcome	Display welcome ASCII art
get_user_info	Read name and birth date
validate_birthdate	Check date validity
determine_zodiac	Identify zodiac sign
display_zodiac_art	Display sign-specific ASCII art
display_menu	Display horoscope choices
show_horoscope	Select and show prediction
exit_program	Terminate program

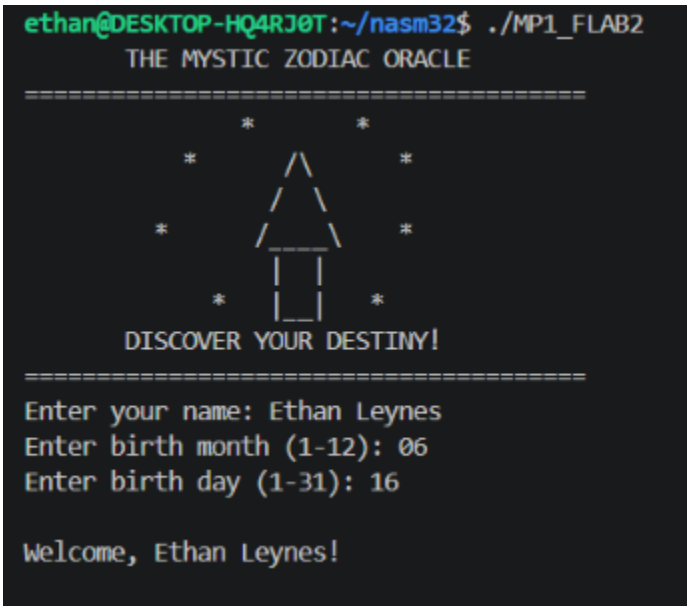
Required branching: CMP, JE/JNE for menus, and JL/JG/JLE/JGE for date ranges; include a menu loop.

Test Cases

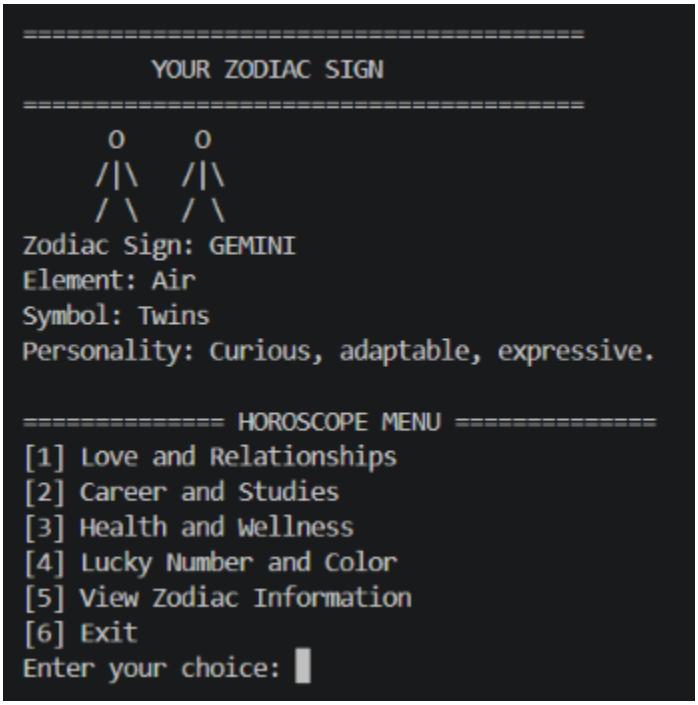
Test	Input	Expected result
1	March 21	Aries
2	April 19	Aries
3	April 20	Taurus
4	June 15	Gemini
5	July 23	Leo
6	August 15	Leo
7	December 25	Capricorn
8	February 30	Invalid date
9	Month 13	Invalid month
10	Menu choice 9	Invalid selection

SCREEN SHOTS:

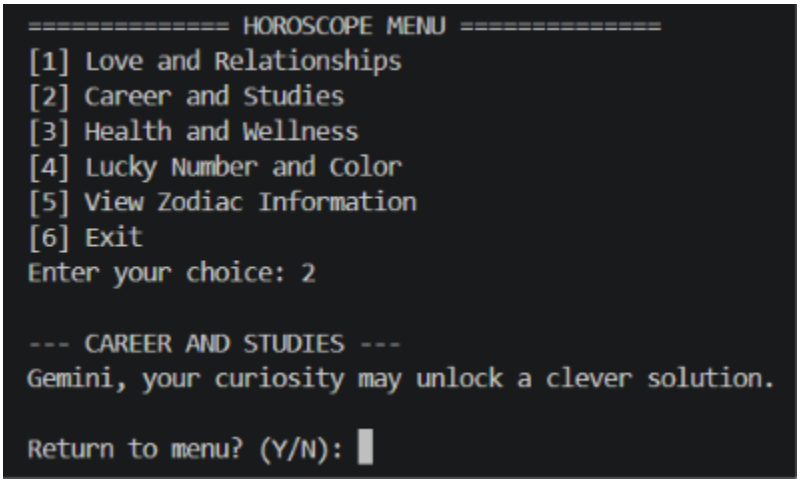
Welcome Screen and User Information



Zodiac Identification and Horoscope Menu



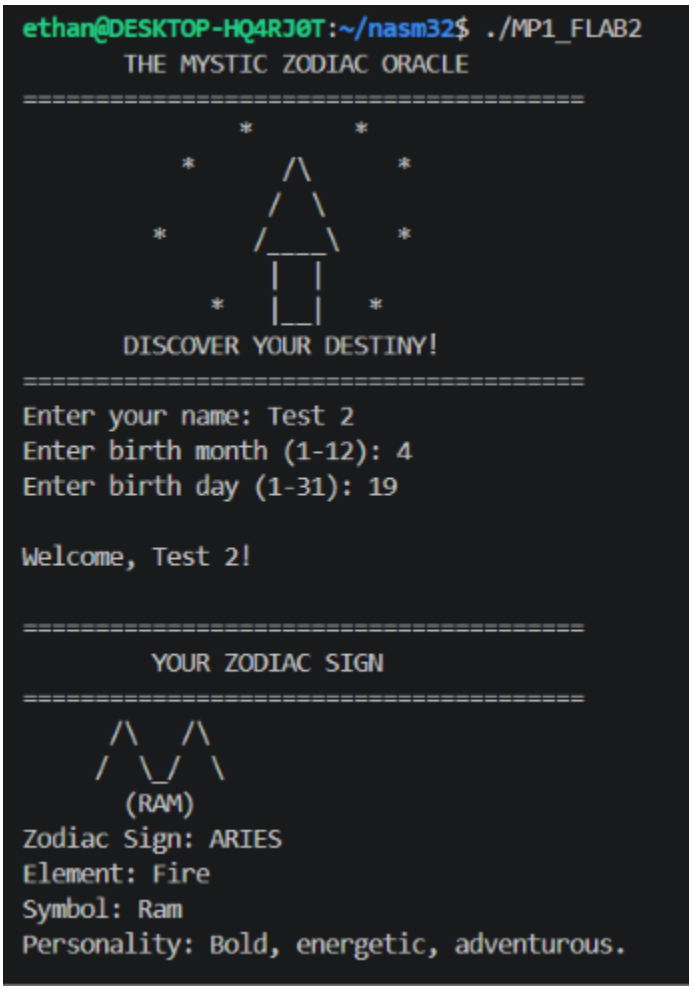
Horoscope Prediction



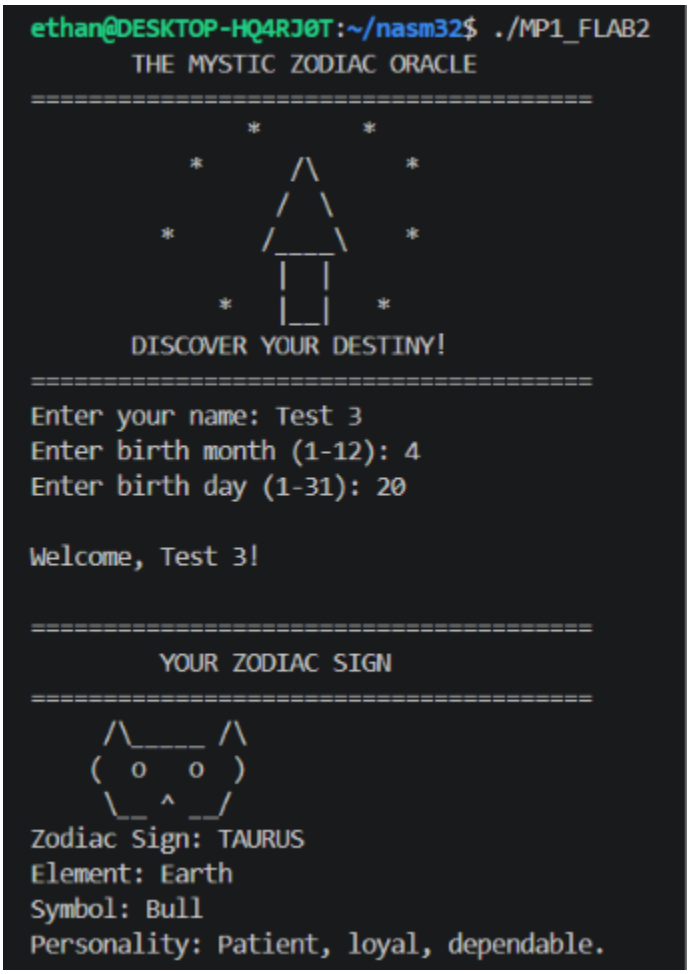
TEST 1


ethan@DESKTOP-HQ4RJ0T:~/nasm32$ ./MP1_FLAB2
THE MYSTIC ZODIAC ORACLE
DISCOVER YOUR DESTINY!
Enter your name: Test 1
Enter birth month (1-12): 3
Enter birth day (1-31): 21
Welcome, Test 1!
YOUR ZODIAC SIGN
(RAM)
Zodiac Sign: ARIES
Element: Fire
Symbol: Ram
Personality: Bold, energetic, adventurous.

TEST 2


ethan@DESKTOP-HQ4RJ0T:~/nasm32$ ./MP1_FLAB2
THE MYSTIC ZODIAC ORACLE
DISCOVER YOUR DESTINY!
Enter your name: Test 2
Enter birth month (1-12): 4
Enter birth day (1-31): 19
Welcome, Test 2!
YOUR ZODIAC SIGN
(RAM)
Zodiac Sign: ARIES
Element: Fire
Symbol: Ram
Personality: Bold, energetic, adventurous.

TEST 3


ethan@DESKTOP-HQ4RJ0T:~/nasm32$ ./MP1_FLAB2
THE MYSTIC ZODIAC ORACLE
DISCOVER YOUR DESTINY!
Enter your name: Test 3
Enter birth month (1-12): 4
Enter birth day (1-31): 20
Welcome, Test 3!
YOUR ZODIAC SIGN
Zodiac Sign: TAURUS
Element: Earth
Symbol: Bull
Personality: Patient, loyal, dependable.

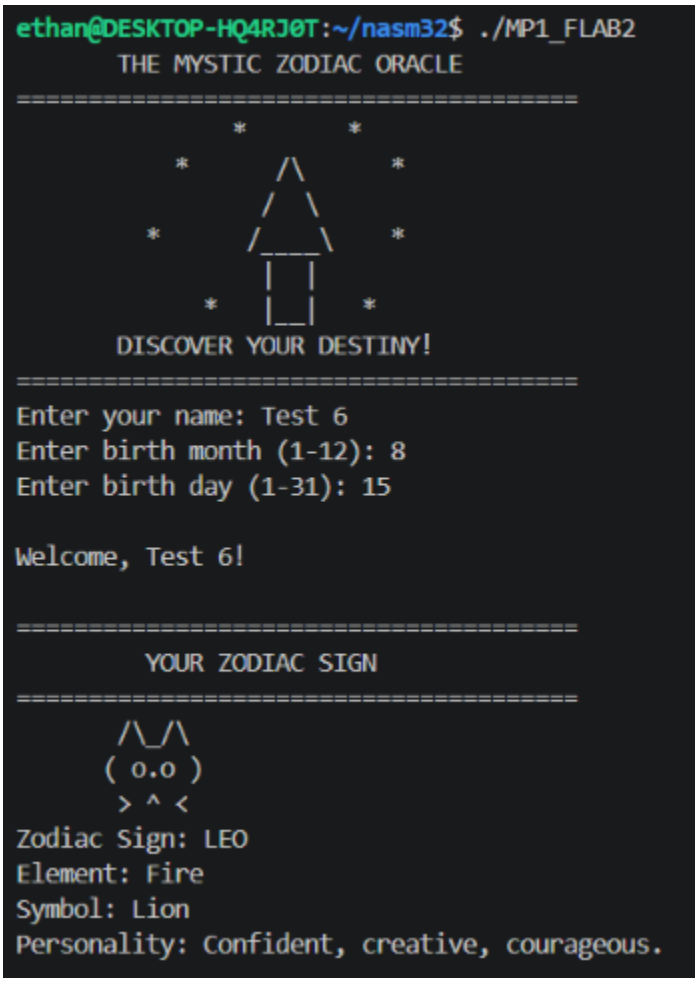
TEST 4


ethan@DESKTOP-HQ4RJ0T:~/nasm32$ ./MP1_FLAB2
THE MYSTIC ZODIAC ORACLE
DISCOVER YOUR DESTINY!
Enter your name: Test 4
Enter birth month (1-12): 6
Enter birth day (1-31): 15
Welcome, Test 4!
YOUR ZODIAC SIGN
Zodiac Sign: GEMINI
Element: Air
Symbol: Twins
Personality: Curious, adaptable, expressive.

TEST 5


ethan@DESKTOP-HQ4RJ0T:~/nasm32$ ./MP1_FLAB2
THE MYSTIC ZODIAC ORACLE
DISCOVER YOUR DESTINY!
Enter your name: Test 5
Enter birth month (1-12): 7
Enter birth day (1-31): 23
Welcome, Test 5 !
YOUR ZODIAC SIGN
Zodiac Sign: LEO
Element: Fire
Symbol: Lion
Personality: Confident, creative, courageous.

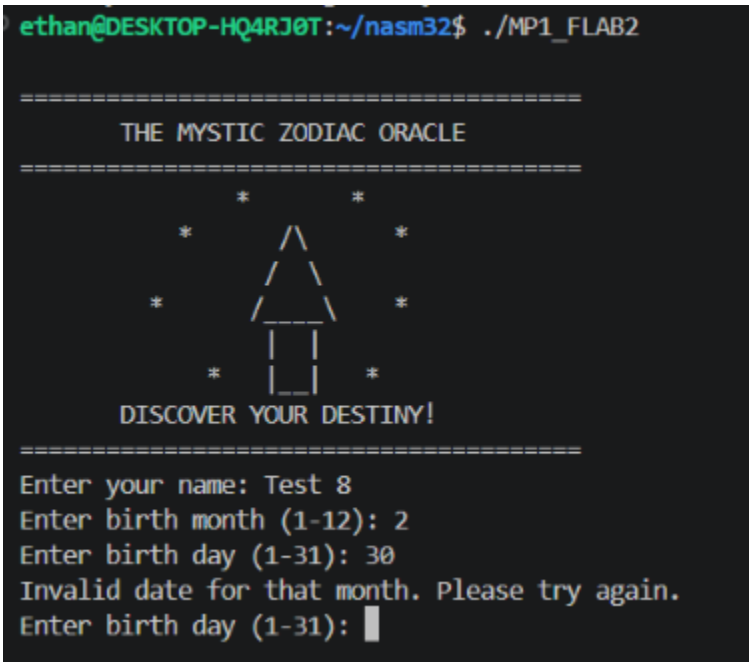
TEST 6


ethan@DESKTOP-HQ4RJ0T:~/nasm32$ ./MP1_FLAB2
THE MYSTIC ZODIAC ORACLE
DISCOVER YOUR DESTINY!
Enter your name: Test 6
Enter birth month (1-12): 8
Enter birth day (1-31): 15
Welcome, Test 6!
YOUR ZODIAC SIGN
Zodiac Sign: LEO
Element: Fire
Symbol: Lion
Personality: Confident, creative, courageous.

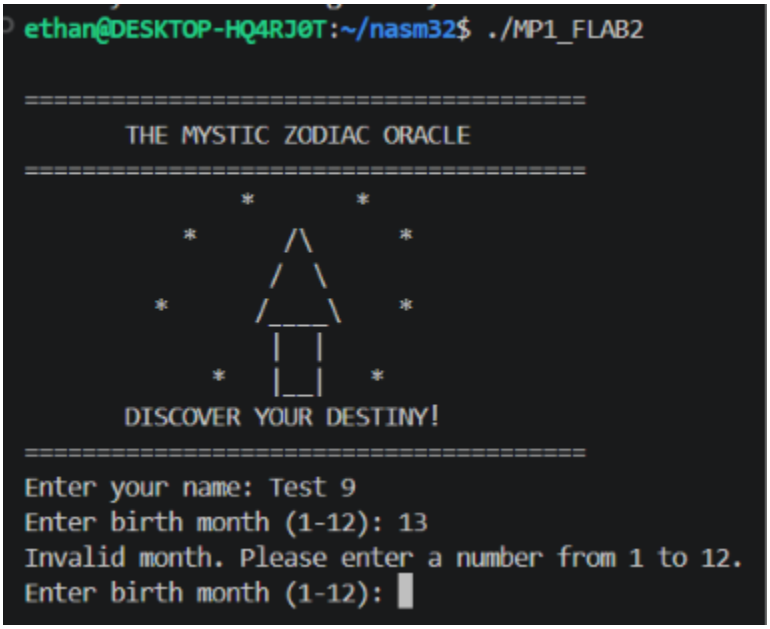
TEST 7



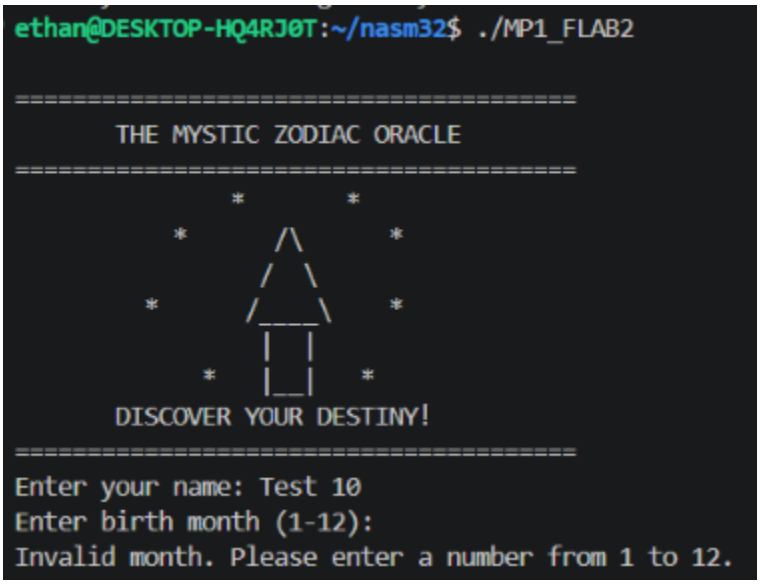
TEST 8 (Invalid Date)



TEST 9 (Invalid Month)



Test 10 (Invalid Menu Choice)



SOURCE CODE:

```
; Module 5 - Machine Problem 1: Zodiac Horoscope and Fortune Teller
; NASM x86 32-bit Linux using int 0x80
; Build: nasm -f elf32 MP1_Zodiac_Horoscope.asm -o MP1_Zodiac_Horoscope.o
; Link:  ld -m elf_i386 MP1_Zodiac_Horoscope.o -o MP1_Zodiac_Horoscope
; Run:   ./MP1_Zodiac_Horoscope

BITS 32
GLOBAL _start

SECTION .data
welcome db 10,'======================================',10
        db '       THE MYSTIC ZODIAC ORACLE',10
        db '======================================',10
        db '             *      *',10
        db '          *     /\     *',10
        db '               /  \',10
```

```
        db '        *      /____\    *',10
        db '               |  |',10
        db '           *   |__|   *',10
        db '       DISCOVER YOUR DESTINY!',10
        db '========================================',10,0
ask_name db 'Enter your name: ',0
ask_month db 'Enter birth month (1-12): ',0
ask_day db 'Enter birth day (1-31): ',0
bad_name db 'Please enter a name (max 63 characters).',10,0
bad_month db 'Invalid month. Please enter a number from 1 to 12.',10,0
bad_day db 'Invalid date for that month. Please try again.',10,0
hello db 10,'Welcome, ',0
bang db '!',10,0
zodiac_header db 10,'======================================',10
            db '         YOUR ZODIAC SIGN',10
            db '======================================',10,0
sign_label db 'Zodiac Sign: ',0
element_label db 'Element: ',0
symbol_label db 'Symbol: ',0
personality_label db 'Personality: ',0
menu db 10,'============== HOROSCOPE MENU ==============',10
    db '[1] Love and Relationships',10
    db '[2] Career and Studies',10
    db '[3] Health and Wellness',10
    db '[4] Lucky Number and Color',10
    db '[5] View Zodiac Information',10
    db '[6] Exit',10
    db 'Enter your choice: ',0
invalid_choice db 'Invalid selection. Choose 1 to 6.',10,0
```

```
return_prompt db 10,'Return to menu? (Y/N): ',0
invalid_yesno db 'Invalid response. Enter Y or N.',10,0
bye db 10,'Thank you for visiting the Mystic Zodiac Oracle!',10,0
love_head db 10,'--- LOVE AND RELATIONSHIPS ---',10,0
career_head db 10,'--- CAREER AND STUDIES ---',10,0
health_head db 10,'--- HEALTH AND WELLNESS ---',10,0
lucky_head db 10,'--- LUCKY NUMBER AND COLOR ---',10,0
lucky_number_label db 'Lucky Number: ',0
lucky_color_label db 'Lucky Color: ',0
nl db 10,0

; January to December number of days (non-leap year; February 29 rejected).
month_days db 31,28,31,30,31,30,31,31,30,31,30,31
; First day of the SECOND zodiac sign in each calendar month.
cutoff_days db 20,19,21,20,21,21,23,23,23,23,22,22
; Zodiac index: 0 Aries, 1 Taurus, 2 Gemini, 3 Cancer, 4 Leo,
; 5 Virgo, 6 Libra, 7 Scorpio, 8 Sagittarius, 9 Capricorn,
; 10 Aquarius, 11 Pisces.
before_sign db 9,10,11,0,1,2,3,4,5,6,7,8
after_sign  db 10,11,0,1,2,3,4,5,6,7,8,9

s0 db 'ARIES',0
s1 db 'TAURUS',0
s2 db 'GEMINI',0
s3 db 'CANCER',0
s4 db 'LEO',0
s5 db 'VIRGO',0
s6 db 'LIBRA',0
s7 db 'SCORPIO',0
```

```
s8 db 'SAGITTARIUS',0

s9 db 'CAPRICORN',0

s10 db 'AQUARIUS',0

s11 db 'PISCES',0

sign_names dd s0,s1,s2,s3,s4,s5,s6,s7,s8,s9,s10,s11


fire db 'Fire',0

earth db 'Earth',0

air db 'Air',0

water db 'Water',0

elements dd fire,earth,air,water,fire,earth,air,water,fire,earth,air,water

sy0 db 'Ram',0

sy1 db 'Bull',0

sy2 db 'Twins',0

sy3 db 'Crab',0

sy4 db 'Lion',0

sy5 db 'Maiden',0

sy6 db 'Scales',0

sy7 db 'Scorpion',0

sy8 db 'Archer',0

sy9 db 'Sea-Goat',0

sy10 db 'Water Bearer',0

sy11 db 'Fish',0

symbols dd sy0,sy1,sy2,sy3,sy4,sy5,sy6,sy7,sy8,sy9,sy10,sy11

p0 db 'Bold, energetic, adventurous.',0

p1 db 'Patient, loyal, dependable.',0

p2 db 'Curious, adaptable, expressive.',0

p3 db 'Caring, intuitive, protective.',0

p4 db 'Confident, creative, courageous.',0
```

```
p5 db 'Thoughtful, practical, observant.',0
p6 db 'Fair, friendly, diplomatic.',0
p7 db 'Passionate, determined, perceptive.',0
p8 db 'Optimistic, independent, adventurous.',0
p9 db 'Disciplined, ambitious, responsible.',0
p10 db 'Original, open-minded, imaginative.',0
p11 db 'Compassionate, artistic, sensitive.',0
personalities dd p0,p1,p2,p3,p4,p5,p6,p7,p8,p9,p10,p11

art0 db '      /\   /\',10,'     /  \_/  \',10,'       (RAM)',10,0
art1 db '     /\_____ /\',10,'    (  o   o  )',10,'     \__ ^ __/',10,0
art2 db '      O     O',10,'     /|\   /|\',10,'     / \   / \',10,0
art3 db '      \  |  /',10,'    --( o o )--',10,'      /  |  \',10,0
art4 db '       /\_/\',10,'      ( o.o )',10,'       > ^ <',10,0
art5 db '        O',10,'       /|\',10,'       / \',10,0
art6 db '    ___     ___',10,'       \___/',10,'        | |',10,'      __|_|__',10,0
art7 db '      /\__/\',10,'     (  ..  )---<',10,'      \/  \/',10,0
art8 db '       /\',10,'   ----->',10,'       \/    (ARROW)',10,0
art9 db '       /\ /\',10,'      (  V  )',10,'       \___/~~',10,0
art10 db '     __[___]__',10,'       \___/  ~~~',10,'        | |  ~~~',10,0
art11 db '      ><((((',10,'    ((((><',10,'      ><((((',10,0
arts dd art0,art1,art2,art3,art4,art5,art6,art7,art8,art9,art10,art11

; Fictional horoscopes: one prediction for every sign/category combination.
l0 db 'Aries, an honest conversation may spark a new connection.',10,0
l1 db 'Taurus, your steady kindness will make someone feel valued.',10,0
l2 db 'Gemini, a thoughtful message could brighten a relationship.',10,0
l3 db 'Cancer, trust your heart while giving others room to speak.',10,0
l4 db 'Leo, your warmth will help a special friendship grow.',10,0
```

```
l5 db 'Virgo, a small act of care can speak louder than words.',10,0
l6 db 'Libra, listening carefully may bring harmony to your day.',10,0
l7 db 'Scorpio, openness can build trust with someone important.',10,0
l8 db 'Sagittarius, share an adventure with someone you cherish.',10,0
l9 db 'Capricorn, making time for loved ones will feel rewarding.',10,0
l10 db 'Aquarius, an unexpected conversation may inspire you.',10,0
l11 db 'Pisces, your empathy can deepen a meaningful bond.',10,0
loves dd l0,l1,l2,l3,l4,l5,l6,l7,l8,l9,l10,l11
c0 db 'Aries, take initiative on a challenging task today.',10,0
c1 db 'Taurus, patient progress will lead to a satisfying result.',10,0
c2 db 'Gemini, your curiosity may unlock a clever solution.',10,0
c3 db 'Cancer, teamwork will help you tackle a difficult lesson.',10,0
c4 db 'Leo, your creativity will help solve a difficult problem.',10,'Stay focused and confident!',10,0
c5 db 'Virgo, careful planning will make your workload easier.',10,0
c6 db 'Libra, collaboration can bring a fresh perspective.',10,0
c7 db 'Scorpio, persistence will pay off on an important project.',10,0
c8 db 'Sagittarius, learning something new will open a door.',10,0
c9 db 'Capricorn, your discipline will impress your teammates.',10,0
c10 db 'Aquarius, a creative idea deserves a chance to shine.',10,0
c11 db 'Pisces, imagination can make an ordinary task special.',10,0
careers dd c0,c1,c2,c3,c4,c5,c6,c7,c8,c9,c10,c11
h0 db 'Aries, balance your energy with time to rest.',10,0
h1 db 'Taurus, a gentle walk may refresh your mood.',10,0
h2 db 'Gemini, take a screen break and clear your mind.',10,0
h3 db 'Cancer, make time for calm and restorative activities.',10,0
h4 db 'Leo, enjoy some movement and remember to hydrate.',10,0
h5 db 'Virgo, a simple routine may help you feel centered.',10,0
h6 db 'Libra, seek balance between activity and relaxation.',10,0
h7 db 'Scorpio, a peaceful moment may restore your focus.',10,0
```

```
h8 db 'Sagittarius, outdoor time can lift your spirits.',10,0
h9 db 'Capricorn, remember that breaks support productivity.',10,0
h10 db 'Aquarius, try a relaxing hobby after your work.',10,0
h11 db 'Pisces, spend a little time on creative self-care.',10,0
healths dd h0,h1,h2,h3,h4,h5,h6,h7,h8,h9,h10,h11
lucky_numbers db 9,6,5,2,1,7,8,4,3,10,11,12
co0 db 'Red',0
co1 db 'Green',0
co2 db 'Yellow',0
co3 db 'Silver',0
co4 db 'Gold',0
co5 db 'Navy Blue',0
co6 db 'Pink',0
co7 db 'Maroon',0
co8 db 'Purple',0
co9 db 'Brown',0
co10 db 'Turquoise',0
co11 db 'Sea Green',0
colors dd co0,co1,co2,co3,co4,co5,co6,co7,co8,co9,co10,co11

SECTION .bss
name_buf resb 64
input_buf resb 128
birth_month resd 1
birth_day resd 1
zodiac_index resd 1
menu_choice resd 1
num_buf resb 12
```

```
SECTION .text
_start:
    call display_welcome
    call get_user_info
    call determine_zodiac
    call display_zodiac_art
.menu_loop:
    call display_menu
    call read_line
    call parse_uint
    jc .invalid_menu
    cmp eax,1
    jl .invalid_menu
    cmp eax,6
    jg .invalid_menu
    mov [menu_choice],eax
    cmp eax,6
    je exit_program
    cmp eax,5
    je .view_zodiac
    call show_horoscope
    jmp .ask_again
.view_zodiac:
    call display_zodiac_art
.ask_again:
    call ask_return
    cmp eax,1
    je .menu_loop
    jmp exit_program
```

```
.invalid_menu:
    mov ecx,invalid_choice
    call print_string
    jmp .menu_loop

; Display the first ASCII art design.
display_welcome:
    mov ecx,welcome
    call print_string
    ret

; Read a nonempty name and two numeric inputs.
get_user_info:
.name_again:
    mov ecx,ask_name
    call print_string
    call read_line
    cmp eax,0
    je .invalid_name
    cmp eax,63
    jg .invalid_name
    mov esi,input_buf
    mov edi,name_buf
    mov ecx,eax
    rep movsb
    mov byte [edi],0
    jmp .month_again
.invalid_name:
    mov ecx,bad_name
```

```
    call print_string
    jmp .name_again
.month_again:
    mov ecx,ask_month
    call print_string
    call read_line
    call parse_uint
    jc .invalid_month
    cmp eax,1
    jl .invalid_month
    cmp eax,12
    jg .invalid_month
    mov [birth_month],eax
    jmp .day_again
.invalid_month:
    mov ecx,bad_month
    call print_string
    jmp .month_again
.day_again:
    mov ecx,ask_day
    call print_string
    call read_line
    call parse_uint
    jc .invalid_day
    mov [birth_day],eax
    call validate_birthdate
    jc .invalid_day
    mov ecx,hello
    call print_string
```

```
    mov ecx,name_buf
    call print_string
    mov ecx,bang
    call print_string
    ret
.invalid_day:
    mov ecx,bad_day
    call print_string
    jmp .day_again

; Output CF=0 if date is valid; CF=1 otherwise.
validate_birthdate:
    mov eax,[birth_month]
    cmp eax,1
    jl .bad
    cmp eax,12
    jg .bad
    mov ebx,[birth_day]
    cmp ebx,1
    jl .bad
    movzx edx,byte [month_days+eax-1]
    cmp ebx,edx
    jg .bad
    clc
    ret
.bad:
    stc
    ret
```

```
; Find correct zodiac using monthly cutoff plus signed comparisons.
determine_zodiac:
    mov eax,[birth_month]
    dec eax
    mov ebx,[birth_day]
    movzx edx,byte [cutoff_days+eax]
    cmp ebx,edx
    jge .after
    movzx ecx,byte [before_sign+eax]
    jmp .store
.after:
    movzx ecx,byte [after_sign+eax]
.store:
    mov [zodiac_index],ecx
    ret

; Display sign-specific ASCII art and all requested sign details.
display_zodiac_art:
    push ebx
    mov ebx,[zodiac_index]
    mov ecx,zodiac_header
    call print_string
    mov ecx,[arts+ebx*4]
    call print_string
    mov ecx,sign_label
    call print_string
    mov ecx,[sign_names+ebx*4]
    call print_line
    mov ecx,element_label
```

```
    call print_string
    mov ecx,[elements+ebx*4]
    call print_line
    mov ecx,symbol_label
    call print_string
    mov ecx,[symbols+ebx*4]
    call print_line
    mov ecx,personality_label
    call print_string
    mov ecx,[personalities+ebx*4]
    call print_line
    pop ebx
    ret

display_menu:
    mov ecx,menu
    call print_string
    ret

; Uses selected menu option and zodiac index to show fictional predictions.
show_horoscope:
    push ebx
    mov ebx,[zodiac_index]
    mov eax,[menu_choice]
    cmp eax,1
    je .love
    cmp eax,2
    je .career
    cmp eax,3
```

```
    je .health
    cmp eax,4
    je .lucky
    jmp .done
.love:
    mov ecx,love_head
    call print_string
    mov ecx,[loves+ebx*4]
    call print_string
    jmp .done
.career:
    mov ecx,career_head
    call print_string
    mov ecx,[careers+ebx*4]
    call print_string
    jmp .done
.health:
    mov ecx,health_head
    call print_string
    mov ecx,[healths+ebx*4]
    call print_string
    jmp .done
.lucky:
    mov ecx,lucky_head
    call print_string
    mov ecx,lucky_number_label
    call print_string
    movzx eax,byte [lucky_numbers+ebx]
    call print_uint
```

```
    mov ecx,nl
    call print_string
    mov ecx,lucky_color_label
    call print_string
    mov ecx,[colors+ebx*4]
    call print_line
.done:
    pop ebx
    ret

; Accept Y/y or N/n. Returns EAX=1 for Y, 0 for N.
ask_return:
.again:
    mov ecx,return_prompt
    call print_string
    call read_line
    cmp eax,1
    jne .bad
    mov al,[input_buf]
    cmp al,'Y'
    je .yes
    cmp al,'y'
    je .yes
    cmp al,'N'
    je .no
    cmp al,'n'
    je .no
.bad:
    mov ecx,invalid_yesno
```

```
    call print_string
    jmp .again
.yes:
    mov eax,1
    ret
.no:
    xor eax,eax
    ret

exit_program:
    mov ecx,bye
    call print_string
    mov eax,1
    xor ebx,ebx
    int 0x80

; Utility: write null-terminated string at ECX to stdout.
; Preserves all caller registers.
print_string:
    pushad
    mov edi,ecx
    xor edx,edx
.count:
    cmp byte [edi+edx],0
    je .write
    inc edx
    jmp .count
.write:
    mov eax,4
```

```
    mov ebx,1
    int 0x80
    popad
    ret

print_line:
    call print_string
    mov ecx,nl
    call print_string
    ret

; Read a line of text, strip its newline, and return length in EAX.
; One byte is read at a time to avoid mixing consecutive input lines.
read_line:
    push ebx
    push ecx
    push edx
    push esi
    xor esi,esi
.loop:
    mov eax,3
    mov ebx,0
    mov ecx,input_buf
    add ecx,esi
    mov edx,1
    int 0x80
    cmp eax,1
    jne .finish
    mov al,[input_buf+esi]
```

```
    cmp al,10
    je .finish
    cmp al,13
    je .finish
    cmp esi,126
    jae .discard
    inc esi
    jmp .loop
.discard:
    ; Consume remaining characters without writing outside the buffer.
    mov eax,3
    xor ebx,ebx
    mov ecx,input_buf+127
    mov edx,1
    int 0x80
    cmp eax,1
    jne .finish
    cmp byte [input_buf+127],10
    jne .discard
.finish:
    mov byte [input_buf+esi],0
    mov eax,esi
    pop esi
    pop edx
    pop ecx
    pop ebx
    ret

; Strict unsigned decimal conversion. Returns CF=1 if empty/non-digit/overflow.
```

```
parse_uint:
    push esi
    push ebx
    mov esi,input_buf
    xor eax,eax
    cmp byte [esi],0
    je .error
.loop:
    movzx ebx,byte [esi]
    test ebx,ebx
    jz .ok
    cmp ebx,'0'
    jb .error
    cmp ebx,'9'
    ja .error
    sub ebx,'0'
    cmp eax,429496729
    ja .error
    jne .multiply
    cmp ebx,5
    ja .error
.multiply:
    imul eax,eax,10
    add eax,ebx
    inc esi
    jmp .loop
.ok:
    clc
    jmp .end
```

```
.error:
    stc
.end:
    pop ebx
    pop esi
    ret

; Print unsigned number in EAX (for lucky number).
print_uint:
    pushad
    mov edi,num_buf+11
    mov byte [edi],0
    mov ebx,10
.convert:
    xor edx,edx
    div ebx
    add dl,'0'
    dec edi
    mov [edi],dl
    test eax,eax
    jnz .convert
    mov ecx,edi
    call print_string
    popad
    ret
```

Machine Problem 2 — Dungeon Adventure and Survival Game

Problem Statement

Create a text-based Dungeon Adventure and Survival Game. The player selects a class, explores three rooms, battles enemies, collects treasure, and makes decisions affecting survival. Use procedures, conditional selections, arithmetic, loops, user input, and ASCII art.

1. Welcome Screen

```
========================================
       DUNGEON OF THE LOST KING
========================================
               |>>>>
               |
           _   _|_  _
          |;|_|;|_|;|
          \.    .  /
           \:  .  /
            ||:   |
            ||:.  |
            ||:  .|
       ENTER IF YOU DARE!
========================================
```

2. Player Registration

```
Enter your hero name: Arthur
Choose your character:
[1] Warrior
[2] Mage
[3] Archer
Enter choice: 1
Welcome, Warrior Arthur!
```

Character	Health	Attack	Potions
Warrior	100	20	2
Mage	70	30	3
Archer	85	25	2

3. Main Menu

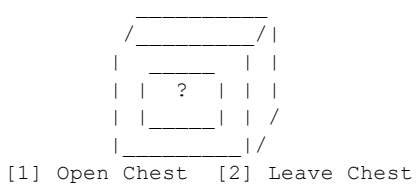
```
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
```

4. Dungeon Rooms and Gameplay

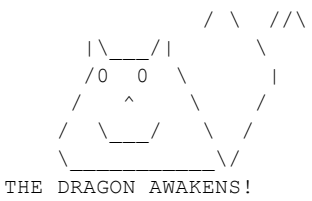
Room 1 — Goblin Chamber: Goblin health = 30. Display enemy ASCII art and offer Attack, Defend, Use Potion, or Run.

```
           ,       ,
          /(.-""-.)\
     |\  \/       \/  /|
     | \ / =.   .= \ / |
      \  \    ^    /  /
       \   '-----'   /
A GOBLIN APPEARS!
[1] Attack  [2] Defend
[3] Use Potion  [4] Run
```

Room 2 — Treasure Chamber: The player chooses to open the chest (gain 50 gold) or leave it (gain nothing).



Room 3 — Dragon's Lair: Dragon health = 60. Show dragon ASCII art and the same combat menu as the goblin.



5. Combat and Outcome Rules

Attack: subtract player attack points from enemy health. Defend: reduce the next enemy attack by 5 (minimum damage 0). Potion: restore 20 health up to starting maximum. Run: leave encounter and return to menu without advancing. A surviving goblin deals 10 damage per turn; a surviving dragon deals 15. After Attack or Defend, the living enemy retaliates. A player loses at health 0 or less; victory occurs after defeating the dragon.

```
VICTORY!
CONGRATULATIONS, ARTHUR!
You defeated the dragon and escaped!
Final Gold: 50
Remaining Health: 35
YOU ARE A HERO!
```

```
GAME OVER
Your hero has fallen.
The dungeon remains undefeated.
BETTER LUCK NEXT TIME!
```

Required Procedures

Procedure	Responsibility
display_welcome	Show title and ASCII art
get_player_info	Read name and class
initialize_character	Assign stats
display_main_menu	Show options
explore_dungeon	Control room progression
battle_enemy	Handle combat selections
use_potion	Restore health
display_status	Show statistics
display_inventory	Show gold and potions
display_game_result	Show victory or defeat
exit_program	Terminate program

Validate class selection, menus, potion count, health boundaries, room progression, and win/loss conditions. Never allow negative displayed health or potions below zero.

Test Cases

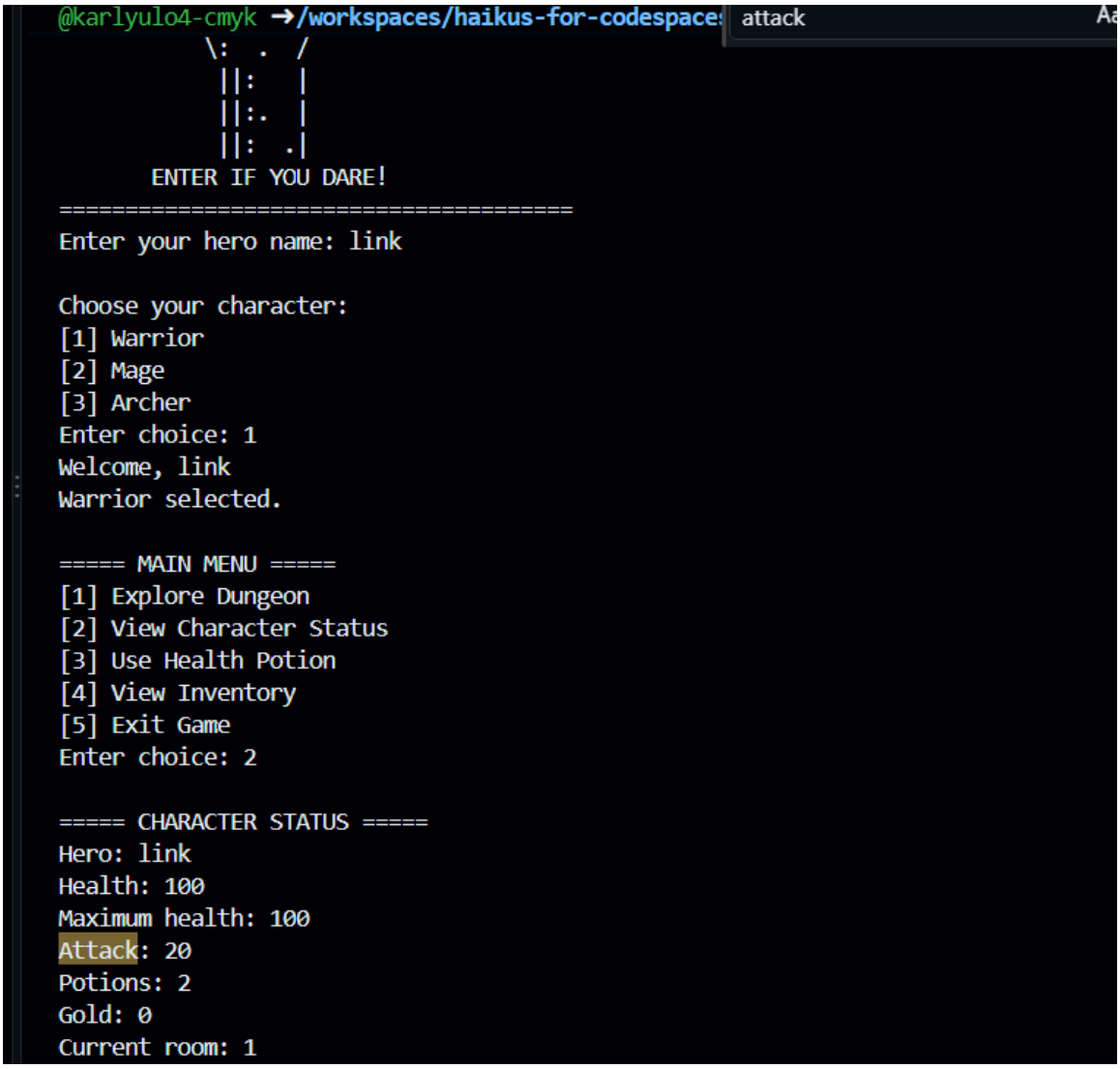
Test	Scenario	Expected result
1	Select Warrior	Health 100, Attack 20
2	Select Mage	Health 70, Attack 30
3	Select Archer	Health 85, Attack 25
4	Select class 9	Invalid selection
5	Warrior attacks Goblin twice	Goblin defeated
6	Potion at full health	Health capped; potion consumed
7	Potion count 0	Error; no health change
8	Open treasure	Gain 50 gold
9	Health reaches 0	Game Over
10	Dragon health reaches 0	Victory screen

Assessment Rubric

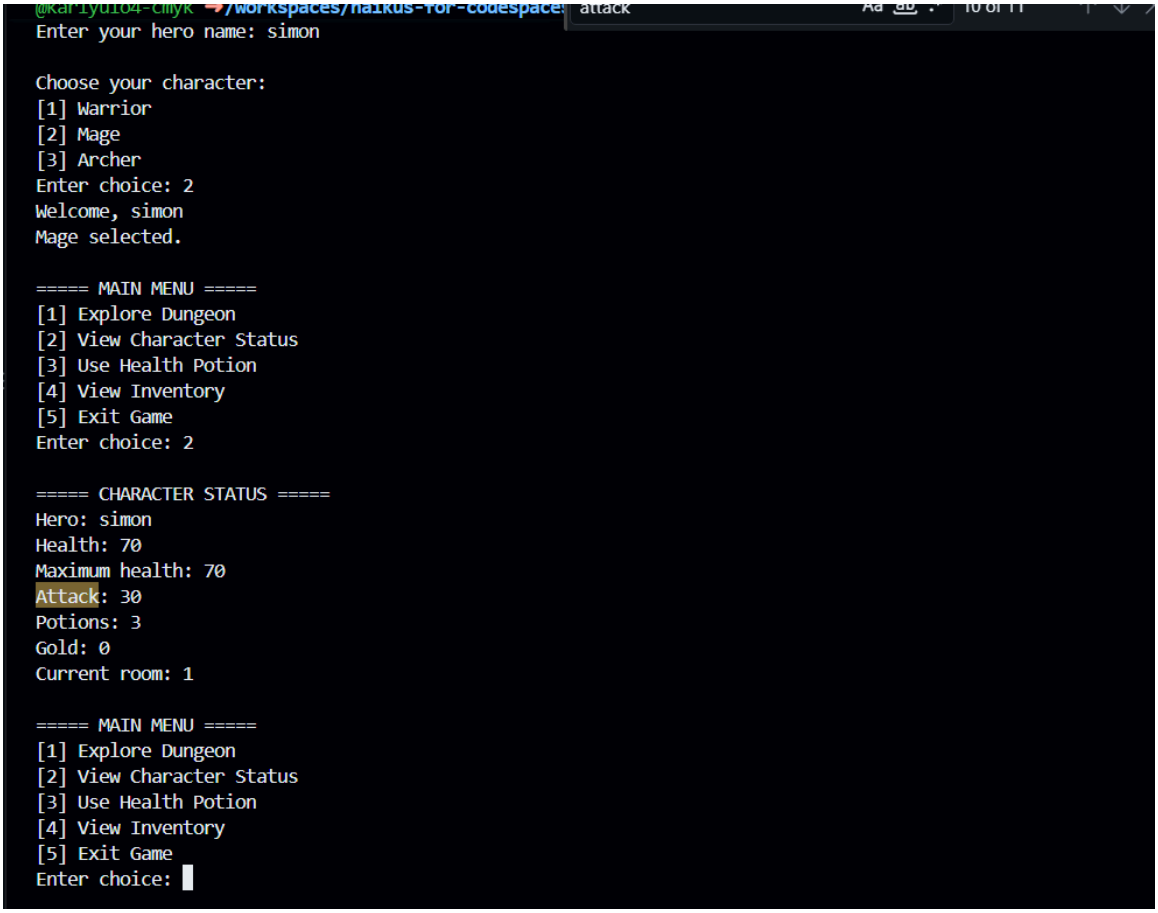
Criterion	Points
Correct procedures (CALL/RET)	25
Conditional selection and branching	20
User input and validation	15
Correct logic and output	15
ASCII art and interface creativity	10
Looping and menu navigation	10
Readability and comments	5
TOTAL	100

Screenshots

Test case 1


@karlyulo4-cmyk →/workspaces/haikus-for-codespace
attack
\: . /
	:
	:.
	: .
ENTER IF YOU DARE!
Enter your hero name: link
Choose your character:
[1] Warrior
[2] Mage
[3] Archer
Enter choice: 1
Welcome, link
Warrior selected.
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice: 2
===== CHARACTER STATUS =====
Hero: link
Health: 100
Maximum health: 100
Attack: 20
Potions: 2
Gold: 0
Current room: 1

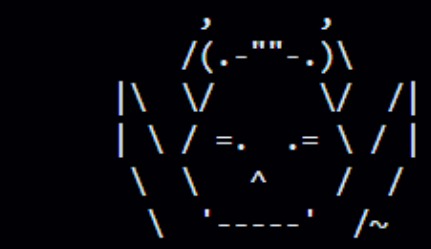
Test case 2:



Test case 3:

```
Gold: 0
Current room: 1

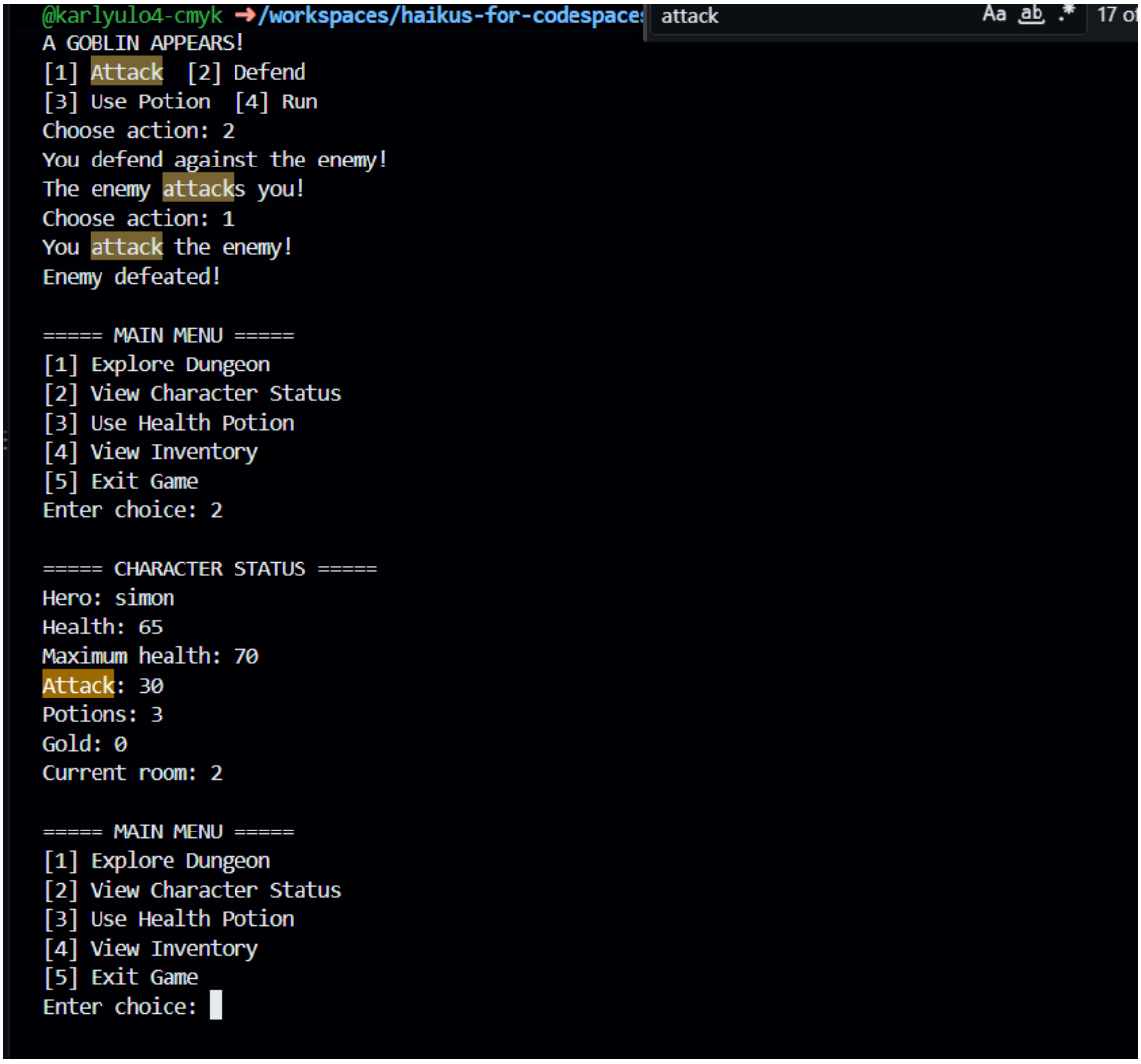
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice: 1
```



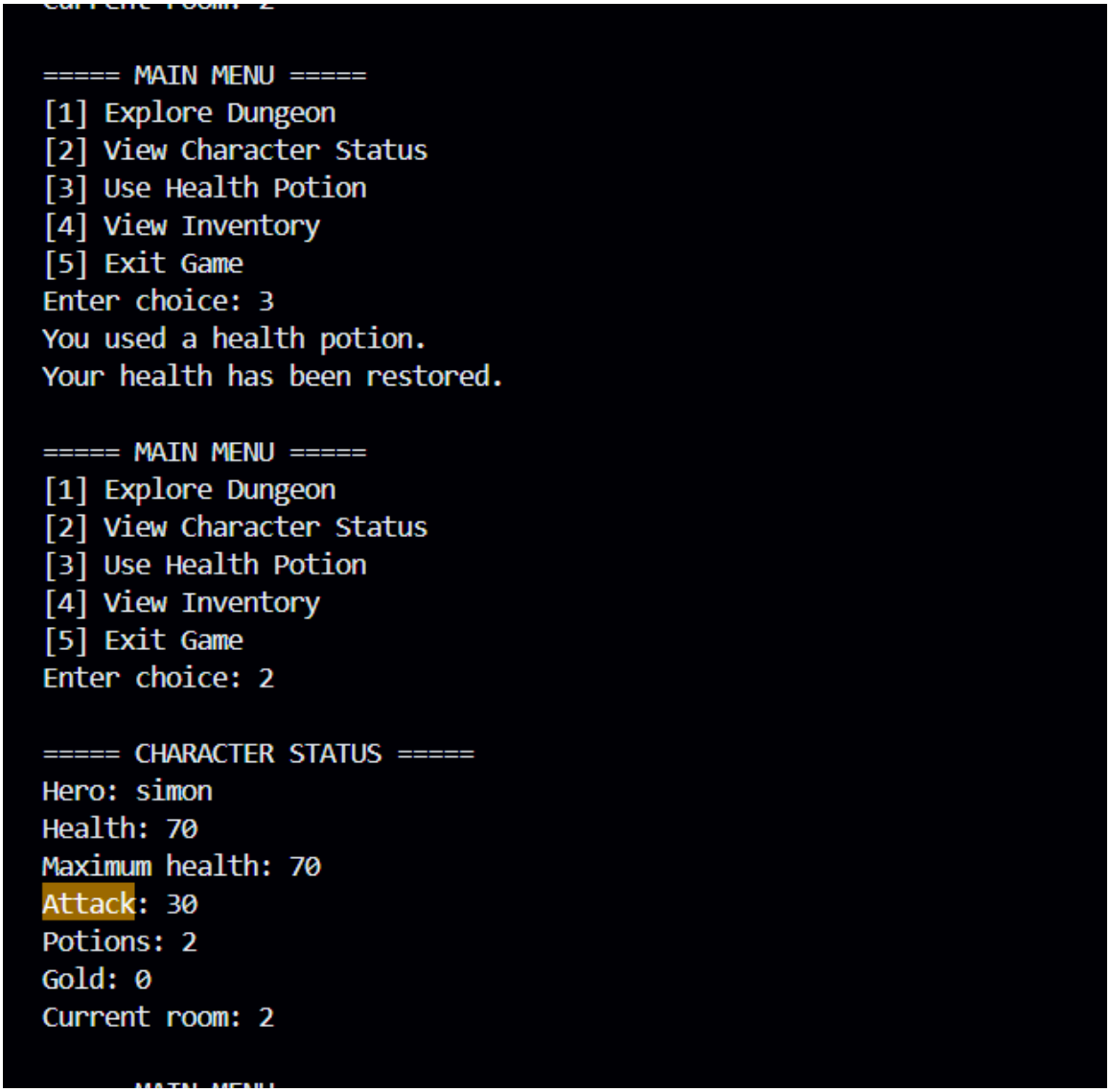
```
A GOBLIN APPEARS!
[1] Attack  [2] Defend
[3] Use Potion  [4] Run
Choose action: 1
You attack the enemy!
Enemy defeated!

===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice:
```

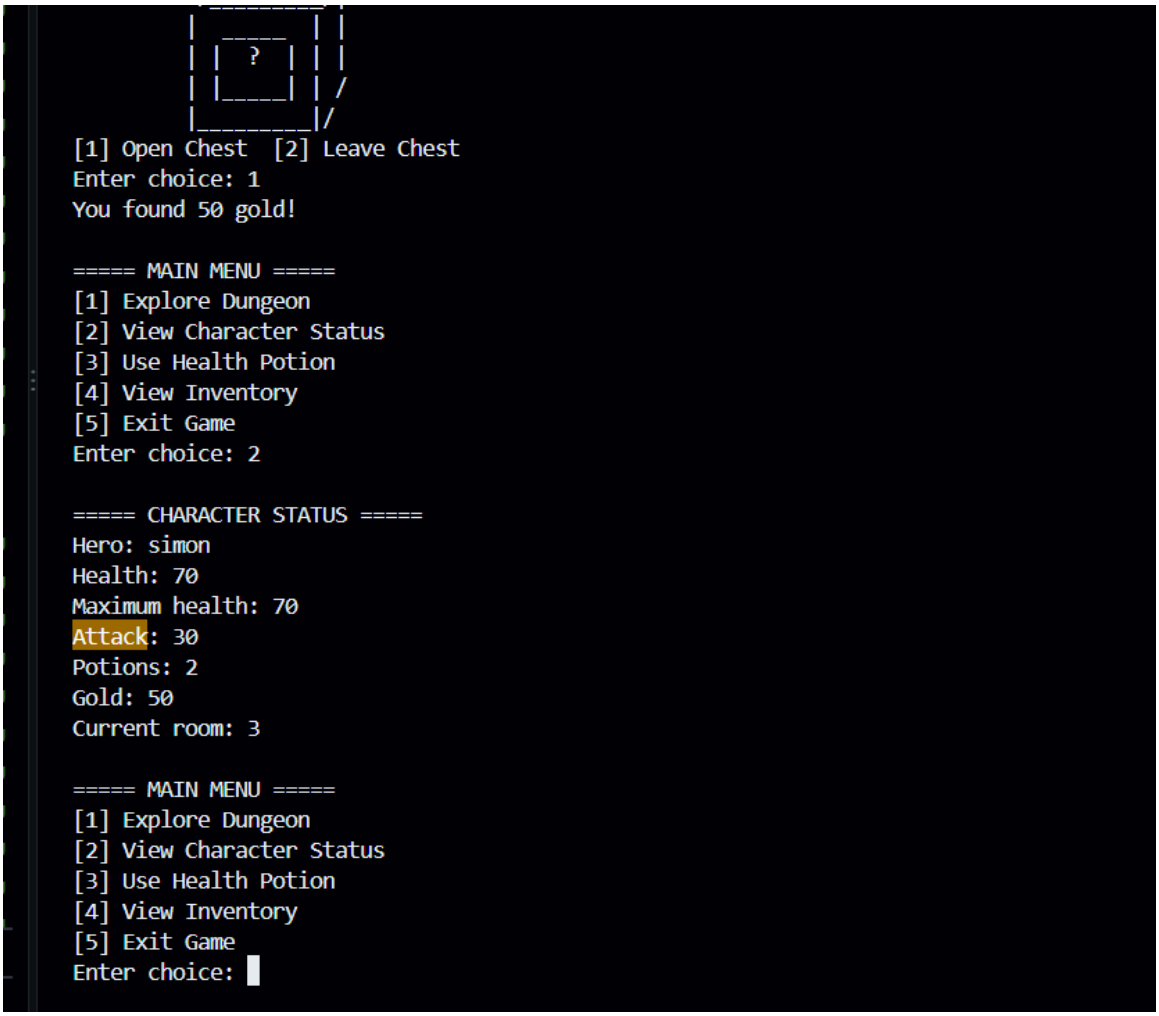
Test case 4:


@karlyulo4-cmyk →/workspaces/haikus-for-codespace
attack
Aa ab .*
17 of
A GOBLIN APPEARS!
[1] Attack [2] Defend
[3] Use Potion [4] Run
Choose action: 2
You defend against the enemy!
The enemy attacks you!
Choose action: 1
You attack the enemy!
Enemy defeated!
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice: 2
===== CHARACTER STATUS =====
Hero: simon
Health: 65
Maximum health: 70
Attack: 30
Potions: 3
Gold: 0
Current room: 2
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice:

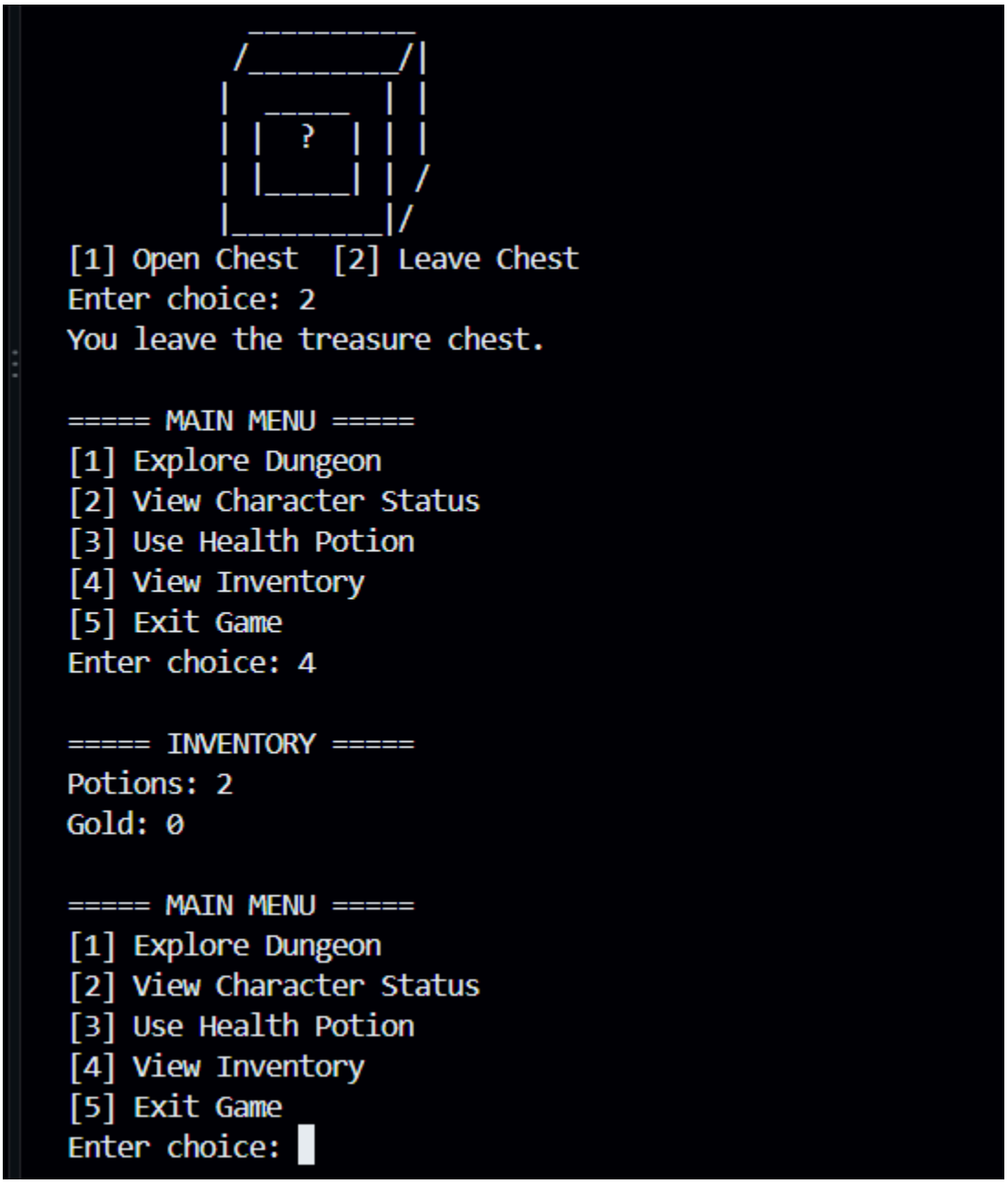
Test case 5:



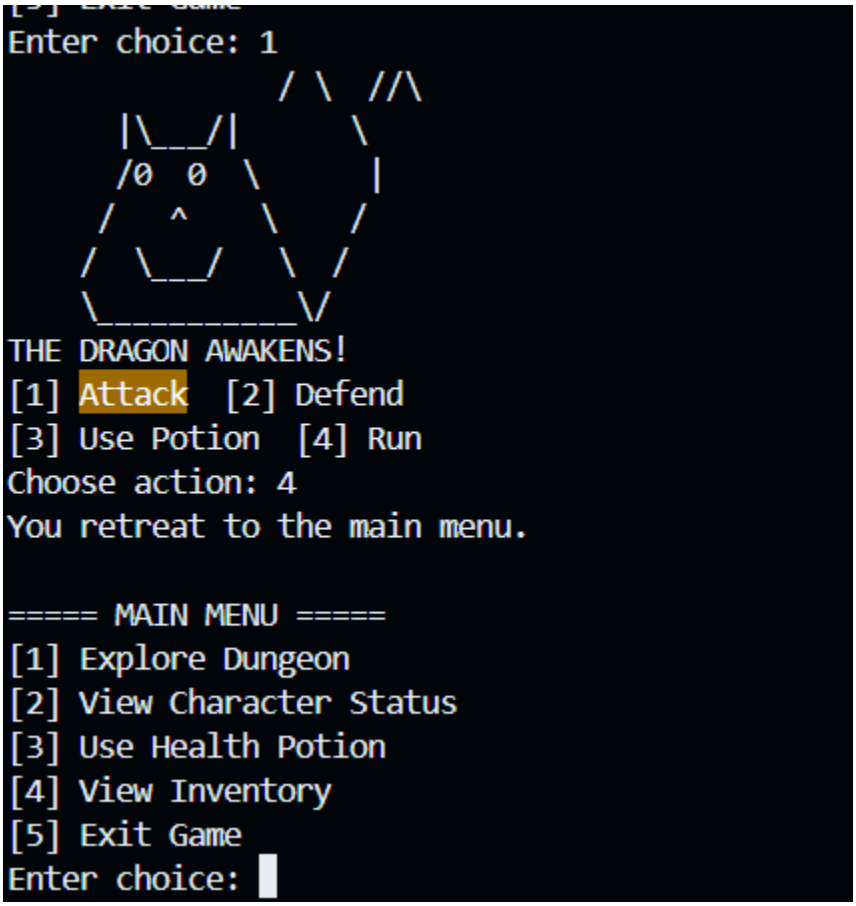
Test case 6:


[1] Open Chest [2] Leave Chest
Enter choice: 1
You found 50 gold!
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice: 2
===== CHARACTER STATUS =====
Hero: simon
Health: 70
Maximum health: 70
Attack: 30
Potions: 2
Gold: 50
Current room: 3
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice:

Test case 7:


[1] Open Chest [2] Leave Chest
Enter choice: 2
You leave the treasure chest.
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice: 4
===== INVENTORY =====
Potions: 2
Gold: 0
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice:

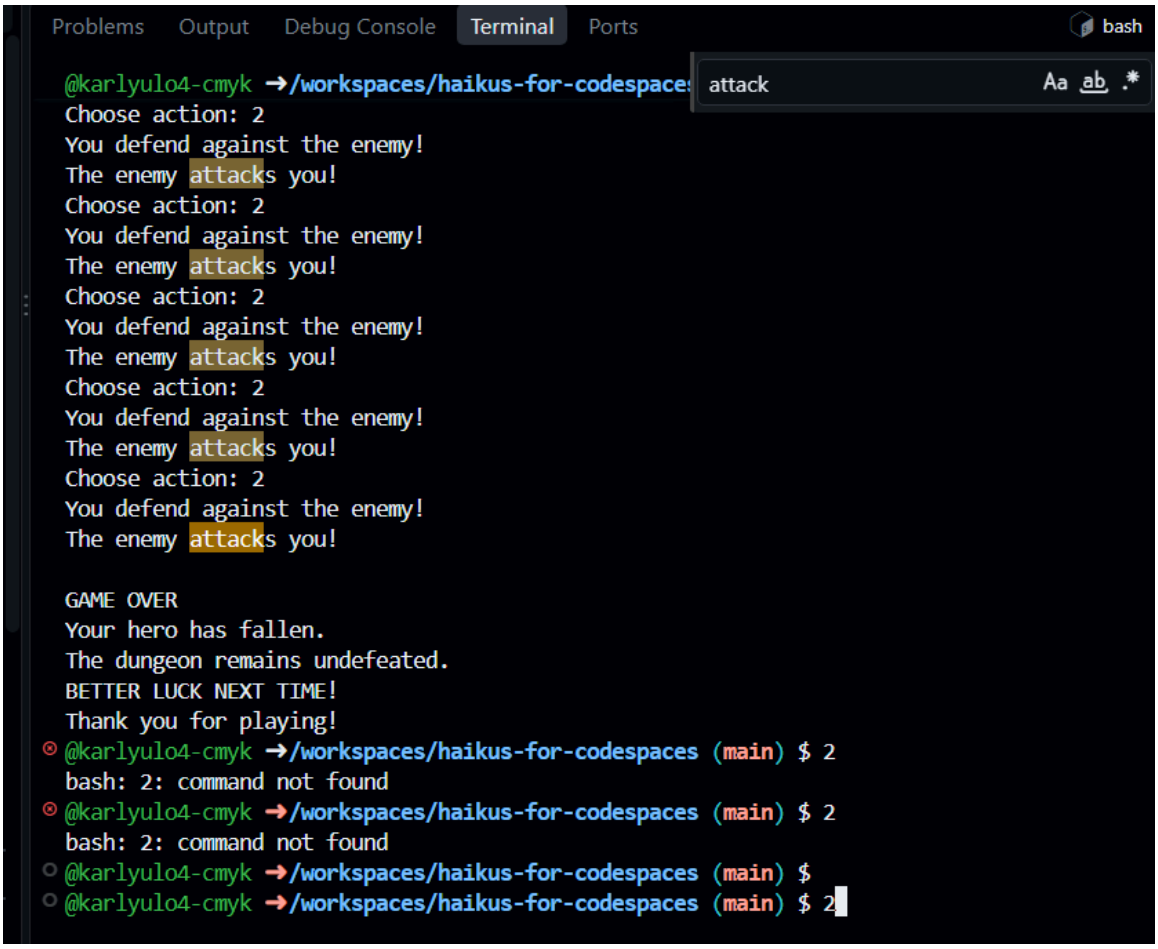
Test case 8:


Enter choice: 1
THE DRAGON AWAKENS!
[1] Attack [2] Defend
[3] Use Potion [4] Run
Choose action: 4
You retreat to the main menu.
===== MAIN MENU =====
[1] Explore Dungeon
[2] View Character Status
[3] Use Health Potion
[4] View Inventory
[5] Exit Game
Enter choice:

Test case 9:


/0 0 \ |
/ ^ \ /
/ __/ \ /
__________\/
THE DRAGON AWAKENS!
[1] Attack [2] Defend
[3] Use Potion [4] Run
Choose action: 1
You attack the enemy!
The enemy attacks you!
Choose action: 1
You attack the enemy!
The enemy attacks you!
Choose action: 1
You attack the enemy!
Enemy defeated!
VICTORY!
Congratulations, hero!
You defeated the dragon and escaped!
Final Gold: 50
Remaining Health: 45
YOU ARE A HERO!
Thank you for playing!
@karlyulo4-cmyk →/workspaces/haikus-for-codespaces (main) $

Test case 10:



Code:

```
section .data


;ascii art

MENU_ART        db"=======================================",0xA
                db"      DUNGEON OF THE LOST KING",0xA
                db"=======================================",0xA
                db"           |>>>>",0xA
                db"           |",0xA
                db"        _ _|_ _",0xA
                db"       |;|_|;|_|;|",0xA
                db"        \.    . /",0xA
                db"         \:  .  /",0xA
                db"          ||:   |",0xA
```

```
            db"           ||:.  |",0xA
            db"           ||:  .|",0xA
            db"       ENTER IF YOU DARE!",0xA
            db"=======================================",0xA
MENU_ART_LEN equ $ - MENU_ART


GOBLIN_ART          db "          ,      ,", 0xA
                db '        /(.-""-.)\', 0xA
                db "     |\  \/      \/  /|", 0xA
                db "     | \ / =.  .= \ / |",0xA
                db "      \  \   ^    /  /",0XA
                db "       \  '-----'  /~",0xA
                db "A GOBLIN APPEARS!",0xA
                db "[1] Attack  [2] Defend",0xA
                db "[3] Use Potion  [4] Run",0xA
  GOBLIN_LEN        equ $ - GOBLIN_ART


CHEST_ART           db "          ___________", 0xA
                db "         /__________/|", 0xA
                db "         |  _____  | |", 0xA
                db "         | |  ?  | | |",0xA
                db "         | |_____| | /",0XA
                db "         |__________|/",0xA
                db "[1] Open Chest  [2] Leave Chest",0xA
  CHEST_LEN         equ $ - CHEST_ART

DRAGON_ART          db "              / \  //\", 0xA
```

```
                db "      |\___/|      \", 0xA
                db "      /0  0  \      |", 0xA
                db "     /   ^    \    /",0xA
                db "    /  \___/   \  /",0XA
                db "    \____________\/",0xA
                db "THE DRAGON AWAKENS!",0xA
                db "[1] Attack  [2] Defend",0xA
                db "[3] Use Potion  [4] Run",0xA
  DRAGON_LEN        equ $ - DRAGON_ART

;player prompts

 NAME_PROMPT db "Enter your hero name: "

 NAME_LEN equ $ - NAME_PROMPT

 CLASS_MENU db 10, "Choose your character:", 10
            db "[1] Warrior", 10
            db "[2] Mage", 10
            db "[3] Archer", 10
            db "Enter choice: "
   CLASS_LEN equ $ - CLASS_MENU

   INVALID_MSG db "Invalid selection. Try again.", 10
   INVALID_LEN equ $ - INVALID_MSG

   WELCOME_PREFIX db "Welcome, "
   WELCOME_PREFIX_LEN equ $ - WELCOME_PREFIX
```

```
    CLASS_WARRIOR_MSG db "Warrior selected.", 10
    CLASS_WARRIOR_LEN equ $ - CLASS_WARRIOR_MSG
    CLASS_MAGE_MSG db "Mage selected.", 10
    CLASS_MAGE_LEN equ $ - CLASS_MAGE_MSG
    CLASS_ARCHER_MSG db "Archer selected.", 10
    CLASS_ARCHER_LEN equ $ - CLASS_ARCHER_MSG



    MAIN_MENU db 10, "===== MAIN MENU =====", 10
            db "[1] Explore Dungeon", 10
            db "[2] View Character Status", 10
            db "[3] Use Health Potion", 10
            db "[4] View Inventory", 10
            db "[5] Exit Game", 10
            db "Enter choice: "
    MAIN_MENU_LEN equ $ - MAIN_MENU

    STATUS_MSG db 10, "Character Status", 10
    STATUS_LEN equ $ - STATUS_MSG

    INVENTORY_MSG db 10, "Inventory", 10
    INVENTORY_LEN equ $ - INVENTORY_MSG

    EXIT_MSG db "Thank you for playing!", 10
    EXIT_LEN equ $ - EXIT_MSG

STATUS_TITLE db 10, "===== CHARACTER STATUS =====", 10
    STATUS_TITLE_LEN equ $ - STATUS_TITLE
```

```
HERO_LABEL db "Hero: "
HERO_LABEL_LEN equ $ - HERO_LABEL

HEALTH_LABEL db "Health: "
HEALTH_LABEL_LEN equ $ - HEALTH_LABEL
ATTACK_LABEL db "Attack: "

ATTACK_LABEL_LEN equ $ - ATTACK_LABEL
POTION_LABEL db "Potions: "

POTION_LABEL_LEN equ $ - POTION_LABEL
GOLD_LABEL db "Gold: "

GOLD_LABEL_LEN equ $ - GOLD_LABEL
ROOM_LABEL db "Current room: "
ROOM_LABEL_LEN equ $ - ROOM_LABEL

MAX_HEALTH_LABEL db "Maximum health: "
MAX_HEALTH_LABEL_LEN equ $ - MAX_HEALTH_LABEL

INVENTORY_TITLE db 10, "===== INVENTORY =====", 10
INVENTORY_TITLE_LEN equ $ - INVENTORY_TITLE

POTION_USED_MSG db "You used a health potion.", 10
POTION_USED_LEN equ $ - POTION_USED_MSG
NO_POTIONS_MSG db "You have no potions remaining!", 10
NO_POTIONS_LEN equ $ - NO_POTIONS_MSG
HEALTH_FULL_MSG db "Your health is already full. Potion consumed.", 10
HEALTH_FULL_LEN equ $ - HEALTH_FULL_MSG
```

```
HEALTH_RESTORED_MSG db "Your health has been restored.", 10
HEALTH_RESTORED_LEN equ $ - HEALTH_RESTORED_MSG

ATTACK_MSG db "You attack the enemy!", 10
ATTACK_MSG_LEN equ $ - ATTACK_MSG

DEFEND_MSG db "You defend against the enemy!", 10
DEFEND_MSG_LEN equ $ - DEFEND_MSG

ENEMY_ATTACK_MSG db "The enemy attacks you!", 10
ENEMY_ATTACK_LEN equ $ - ENEMY_ATTACK_MSG

ENEMY_DEFEATED_MSG db "Enemy defeated!", 10
ENEMY_DEFEATED_LEN equ $ - ENEMY_DEFEATED_MSG

RUN_MSG db "You retreat to the main menu.", 10
RUN_LEN equ $ - RUN_MSG

CHEST_OPEN_MSG db "You found 50 gold!", 10
CHEST_OPEN_LEN equ $ - CHEST_OPEN_MSG

CHEST_LEAVE_MSG db "You leave the treasure chest.", 10
CHEST_LEAVE_LEN equ $ - CHEST_LEAVE_MSG

CHEST_PROMPT db "Enter choice: "
CHEST_PROMPT_LEN equ $ - CHEST_PROMPT

COMBAT_PROMPT db "Choose action: "
COMBAT_PROMPT_LEN equ $ - COMBAT_PROMPT
```

```
    GAME_OVER_MSG db 10, "GAME OVER", 10
                  db "Your hero has fallen.", 10
                  db "The dungeon remains undefeated.", 10
                  db "BETTER LUCK NEXT TIME!", 10
    GAME_OVER_LEN equ $ - GAME_OVER_MSG

    VICTORY_MSG db 10, "VICTORY!", 10
                db "Congratulations, hero!", 10
                db "You defeated the dragon and escaped!", 10
                db "Final Gold: "

    VICTORY_PREFIX_LEN equ $ - VICTORY_MSG

    HEALTH_RESULT_MSG db "Remaining Health: "
    HEALTH_RESULT_LEN equ $ - HEALTH_RESULT_MSG

    HERO_WIN_MSG db "YOU ARE A HERO!", 10
    HERO_WIN_LEN equ $ - HERO_WIN_MSG

    GOODBYE_MSG db "Thank you for playing!", 10
    GOODBYE_LEN equ $ - GOODBYE_MSG

    NEWLINE db 10

section .bss
    hero_name resb 32
    hero_name_len resd 1
```

```
    class_choice resb 2
    menu_choice resb 2
    combat_choice resb 2
    chest_choice resb 2

    player_health resd 1
    max_health resd 1
    player_attack resd 1
    player_potions resd 1
    player_gold resd 1

    enemy_health resd 1
    room_number resd 1
    defending resd 1
    enemy_damage resd 1

    number_buffer resb 12

section .text
    global _start

; ====================================
; PROGRAM ENTRY
; ====================================
_start:
    call display_welcome
    call get_player_info
    call initialize_character
    call display_main_menu
```

```
; ====================================
; EXIT PROGRAM
; ====================================
exit_program:
    mov eax, 4
    mov ebx, 1
    mov ecx, GOODBYE_MSG
    mov edx, GOODBYE_LEN
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80

; ====================================
; DISPLAY WELCOME SCREEN
; ====================================
display_welcome:
    mov eax, 4
    mov ebx, 1
    mov ecx, MENU_ART
    mov edx, MENU_ART_LEN
    int 0x80
    ret

; ====================================
; GET PLAYER NAME AND CLASS
; ====================================
```

```
get_player_info:
    mov eax, 4
    mov ebx, 1
    mov ecx, NAME_PROMPT
    mov edx, NAME_LEN
    int 0x80

    mov eax, 3
    mov ebx, 0
    mov ecx, hero_name
    mov edx, 32
    int 0x80
    mov [hero_name_len], eax

get_class:
    mov eax, 4
    mov ebx, 1
    mov ecx, CLASS_MENU
    mov edx, CLASS_LEN
    int 0x80

    mov eax, 3
    mov ebx, 0
    mov ecx, class_choice
    mov edx, 2
    int 0x80

    mov al, [class_choice]
    cmp al, '1'
```

```
    je class_input_valid
    cmp al, '2'
    je class_input_valid
    cmp al, '3'
    je class_input_valid

    call show_invalid
    jmp get_class

class_input_valid:
    ret

; ====================================
; INITIALIZE CHARACTER STATISTICS
; ====================================
initialize_character:
    mov al, [class_choice]

    cmp al, '1'
    je initialize_warrior
    cmp al, '2'
    je initialize_mage
    cmp al, '3'
    je initialize_archer
    ret

initialize_warrior:
    mov dword [player_health], 100
    mov dword [max_health], 100
```

```
    mov dword [player_attack], 20
    mov dword [player_potions], 2
    jmp character_initialized

initialize_mage:
    mov dword [player_health], 70
    mov dword [max_health], 70
    mov dword [player_attack], 30
    mov dword [player_potions], 3
    jmp character_initialized

initialize_archer:
    mov dword [player_health], 85
    mov dword [max_health], 85
    mov dword [player_attack], 25
    mov dword [player_potions], 2

character_initialized:
    mov dword [player_gold], 0
    mov dword [room_number], 1
    mov dword [defending], 0

    mov eax, 4
    mov ebx, 1
    mov ecx, WELCOME_PREFIX
    mov edx, WELCOME_PREFIX_LEN
    int 0x80

    mov eax, 4
```

```
    mov ebx, 1
    mov ecx, hero_name
    mov edx, [hero_name_len]
    int 0x80

    mov al, [class_choice]
    cmp al, '1'
    je print_warrior_class
    cmp al, '2'
    je print_mage_class

    mov eax, 4
    mov ebx, 1
    mov ecx, CLASS_ARCHER_MSG
    mov edx, CLASS_ARCHER_LEN
    int 0x80
    ret

print_warrior_class:
    mov eax, 4
    mov ebx, 1
    mov ecx, CLASS_WARRIOR_MSG
    mov edx, CLASS_WARRIOR_LEN
    int 0x80
    ret

print_mage_class:
    mov eax, 4
    mov ebx, 1
```

```
    mov ecx, CLASS_MAGE_MSG
    mov edx, CLASS_MAGE_LEN
    int 0x80
    ret

; ====================================
; MAIN MENU LOOP
; ====================================
display_main_menu:
menu_loop:
    mov eax, 4
    mov ebx, 1
    mov ecx, MAIN_MENU
    mov edx, MAIN_MENU_LEN
    int 0x80

    mov eax, 3
    mov ebx, 0
    mov ecx, menu_choice
    mov edx, 2
    int 0x80

    mov al, [menu_choice]

    cmp al, '1'
    je explore_dungeon
    cmp al, '2'
    je display_status
    cmp al, '3'
```

```
    je main_use_potion
    cmp al, '4'
    je display_inventory
    cmp al, '5'
    je exit_program

    call show_invalid
    jmp menu_loop

; ====================================
; EXPLORE THE CURRENT ROOM
; Room 1 = Goblin, Room 2 = Chest,
; Room 3 = Dragon
; ====================================
explore_dungeon:
    cmp dword [room_number], 1
    je explore_goblin
    cmp dword [room_number], 2
    je explore_chest
    cmp dword [room_number], 3
    je explore_dragon

    jmp menu_loop

explore_goblin:
    mov dword [enemy_health], 30
    mov dword [enemy_damage], 10
    call display_goblin
    call battle_enemy
```

```
    cmp eax, 0
    je game_over
    cmp eax, 2
    je menu_loop

    mov dword [room_number], 2
    jmp menu_loop

explore_chest:
chest_choice_loop:
    call display_chest

    mov eax, 4
    mov ebx, 1
    mov ecx, CHEST_PROMPT
    mov edx, CHEST_PROMPT_LEN
    int 0x80

    mov eax, 3
    mov ebx, 0
    mov ecx, chest_choice
    mov edx, 2
    int 0x80

    mov al, [chest_choice]
    cmp al, '1'
    je open_chest
    cmp al, '2'
```

```
    je leave_chest

    call show_invalid
    jmp chest_choice_loop

open_chest:
    add dword [player_gold], 50

    mov eax, 4
    mov ebx, 1
    mov ecx, CHEST_OPEN_MSG
    mov edx, CHEST_OPEN_LEN
    int 0x80

    mov dword [room_number], 3
    jmp menu_loop

leave_chest:
    mov eax, 4
    mov ebx, 1
    mov ecx, CHEST_LEAVE_MSG
    mov edx, CHEST_LEAVE_LEN
    int 0x80

    mov dword [room_number], 3
    jmp menu_loop

explore_dragon:
    mov dword [enemy_health], 60
```

```
    mov dword [enemy_damage], 15
    call display_dragon
    call battle_enemy

    cmp eax, 0
    je game_over
    cmp eax, 2
    je menu_loop

    jmp victory

; ====================================
; DISPLAY ROOM ART
; ====================================
display_goblin:
    mov eax, 4
    mov ebx, 1
    mov ecx, GOBLIN_ART
    mov edx, GOBLIN_LEN
    int 0x80
    ret

display_chest:
    mov eax, 4
    mov ebx, 1
    mov ecx, CHEST_ART
    mov edx, CHEST_LEN
    int 0x80
    ret
```

```
display_dragon:
    mov eax, 4
    mov ebx, 1
    mov ecx, DRAGON_ART
    mov edx, DRAGON_LEN
    int 0x80
    ret

; ====================================
; BATTLE ENEMY
; Return EAX:
; 0 = player defeated
; 1 = enemy defeated
; 2 = player ran away
; ====================================
battle_enemy:
combat_loop:
    mov eax, 4
    mov ebx, 1
    mov ecx, COMBAT_PROMPT
    mov edx, COMBAT_PROMPT_LEN
    int 0x80

    mov eax, 3
    mov ebx, 0
    mov ecx, combat_choice
    mov edx, 2
    int 0x80
```

```
    mov al, [combat_choice]

    cmp al, '1'
    je combat_attack
    cmp al, '2'
    je combat_defend
    cmp al, '3'
    je combat_potion
    cmp al, '4'
    je combat_run

    call show_invalid
    jmp combat_loop

combat_attack:
    mov eax, 4
    mov ebx, 1
    mov ecx, ATTACK_MSG
    mov edx, ATTACK_MSG_LEN
    int 0x80

    mov eax, [player_attack]
    sub [enemy_health], eax

    cmp dword [enemy_health], 0
    jle enemy_defeated

    ; A living enemy retaliates after an attack.
```

```
    call enemy_retaliates
    cmp dword [player_health], 0
    jle player_defeated

    jmp combat_loop

combat_defend:
    mov dword [defending], 1

    mov eax, 4
    mov ebx, 1
    mov ecx, DEFEND_MSG
    mov edx, DEFEND_MSG_LEN
    int 0x80

    ; Defend reduces the next attack by 5.
    call enemy_retaliates
    cmp dword [player_health], 0
    jle player_defeated

    jmp combat_loop

combat_potion:
    call use_potion
    cmp dword [player_health], 0
    jle player_defeated
    ; The assignment specifies enemy retaliation after
    ; Attack or Defend, so potion use does not trigger it.
    jmp combat_loop
```

```
combat_run:
    mov eax, 4
    mov ebx, 1
    mov ecx, RUN_MSG
    mov edx, RUN_LEN
    int 0x80

    mov eax, 2
    ret

enemy_retaliates:
    mov eax, 4
    mov ebx, 1
    mov ecx, ENEMY_ATTACK_MSG
    mov edx, ENEMY_ATTACK_LEN
    int 0x80

    mov eax, [enemy_damage]

    cmp dword [defending], 1
    jne apply_enemy_damage

    sub eax, 5
    cmp eax, 0
    jge apply_enemy_damage
    xor eax, eax

apply_enemy_damage:
```

```
    mov dword [defending], 0
    sub [player_health], eax

    cmp dword [player_health], 0
    jg enemy_attack_done
    mov dword [player_health], 0

enemy_attack_done:
    ret

enemy_defeated:
    mov eax, 4
    mov ebx, 1
    mov ecx, ENEMY_DEFEATED_MSG
    mov edx, ENEMY_DEFEATED_LEN
    int 0x80

    mov eax, 1
    ret

player_defeated:
    mov dword [player_health], 0
    xor eax, eax
    ret

; Main-menu potion option: use potion, then return to menu.
main_use_potion:
    call use_potion
    jmp menu_loop
```

```
; ====================================
; USE HEALTH POTION
; Restores 20 health, capped at maximum.
; Consumes a potion even at full health.
; ====================================
use_potion:
    cmp dword [player_potions], 0
    jle no_potions

    dec dword [player_potions]

    mov eax, 4
    mov ebx, 1
    mov ecx, POTION_USED_MSG
    mov edx, POTION_USED_LEN
    int 0x80

    mov eax, [player_health]
    cmp eax, [max_health]
    jge potion_at_full_health

    add eax, 20
    cmp eax, [max_health]
    jle save_potion_health

    mov eax, [max_health]

save_potion_health:
```

```
    mov [player_health], eax

    mov eax, 4
    mov ebx, 1
    mov ecx, HEALTH_RESTORED_MSG
    mov edx, HEALTH_RESTORED_LEN
    int 0x80
    jmp potion_done

potion_at_full_health:
    mov eax, 4
    mov ebx, 1
    mov ecx, HEALTH_FULL_MSG
    mov edx, HEALTH_FULL_LEN
    int 0x80

potion_done:
    ; Return to the caller. Combat and the main menu
    ; decide where execution continues.
    ret

no_potions:
    mov eax, 4
    mov ebx, 1
    mov ecx, NO_POTIONS_MSG
    mov edx, NO_POTIONS_LEN
    int 0x80
    ret
```

```
; ====================================
; DISPLAY CHARACTER STATUS
; ====================================
display_status:
    mov eax, 4
    mov ebx, 1
    mov ecx, STATUS_TITLE
    mov edx, STATUS_TITLE_LEN
    int 0x80

    mov eax, 4
    mov ebx, 1
    mov ecx, HERO_LABEL
    mov edx, HERO_LABEL_LEN
    int 0x80

    mov eax, 4
    mov ebx, 1
    mov ecx, hero_name
    mov edx, [hero_name_len]
    int 0x80

    mov eax, 4
    mov ebx, 1
    mov ecx, HEALTH_LABEL
    mov edx, HEALTH_LABEL_LEN
    int 0x80
    mov eax, [player_health]
    call print_uint
```

```
call print_newline

mov eax, 4
mov ebx, 1
mov ecx, MAX_HEALTH_LABEL
mov edx, MAX_HEALTH_LABEL_LEN
int 0x80
mov eax, [max_health]
call print_uint
call print_newline

mov eax, 4
mov ebx, 1
mov ecx, ATTACK_LABEL
mov edx, ATTACK_LABEL_LEN
int 0x80
mov eax, [player_attack]
call print_uint
call print_newline

mov eax, 4
mov ebx, 1
mov ecx, POTION_LABEL
mov edx, POTION_LABEL_LEN
int 0x80
mov eax, [player_potions]
call print_uint
call print_newline
```

```
    mov eax, 4
    mov ebx, 1
    mov ecx, GOLD_LABEL
    mov edx, GOLD_LABEL_LEN
    int 0x80
    mov eax, [player_gold]
    call print_uint
    call print_newline

    mov eax, 4
    mov ebx, 1
    mov ecx, ROOM_LABEL
    mov edx, ROOM_LABEL_LEN
    int 0x80
    mov eax, [room_number]
    call print_uint
    call print_newline

    jmp menu_loop

; ====================================
; DISPLAY INVENTORY
; ====================================
display_inventory:
    mov eax, 4
    mov ebx, 1
    mov ecx, INVENTORY_TITLE
    mov edx, INVENTORY_TITLE_LEN
    int 0x80
```

```
    mov eax, 4
    mov ebx, 1
    mov ecx, POTION_LABEL
    mov edx, POTION_LABEL_LEN
    int 0x80
    mov eax, [player_potions]
    call print_uint
    call print_newline

    mov eax, 4
    mov ebx, 1
    mov ecx, GOLD_LABEL
    mov edx, GOLD_LABEL_LEN
    int 0x80
    mov eax, [player_gold]
    call print_uint
    call print_newline

    jmp menu_loop

; ====================================
; DISPLAY GAME RESULT
; ====================================
game_over:
    mov eax, 4
    mov ebx, 1
    mov ecx, GAME_OVER_MSG
    mov edx, GAME_OVER_LEN
```

```
    int 0x80
    jmp exit_program

victory:
    mov eax, 4
    mov ebx, 1
    mov ecx, VICTORY_MSG
    mov edx, VICTORY_PREFIX_LEN
    int 0x80

    mov eax, [player_gold]
    call print_uint
    call print_newline

    mov eax, 4
    mov ebx, 1
    mov ecx, HEALTH_RESULT_MSG
    mov edx, HEALTH_RESULT_LEN
    int 0x80

    mov eax, [player_health]
    call print_uint
    call print_newline

    mov eax, 4
    mov ebx, 1
    mov ecx, HERO_WIN_MSG
    mov edx, HERO_WIN_LEN
    int 0x80
```

```
    jmp exit_program

; ====================================
; PRINT INVALID INPUT MESSAGE
; ====================================
show_invalid:
    mov eax, 4
    mov ebx, 1
    mov ecx, INVALID_MSG
    mov edx, INVALID_LEN
    int 0x80
    ret

; ====================================
; PRINT UNSIGNED INTEGER IN EAX
; Preserves registers with PUSHAD/POPAD.
; ====================================
print_uint:
    pushad

    mov edi, number_buffer
    add edi, 11
    mov byte [edi], 0
    mov ecx, 0

    mov ebx, 10
    cmp eax, 0
    jne convert_number
```

```
    dec edi
    mov byte [edi], '0'
    mov ecx, 1
    jmp output_number

convert_number:
    xor edx, edx
    div ebx
    add dl, '0'
    dec edi
    mov [edi], dl
    inc ecx
    test eax, eax
    jnz convert_number

output_number:
    mov eax, 4
    mov ebx, 1
    mov edx, ecx
    mov ecx, edi
    int 0x80

    popad
    ret

print_newline:
    mov eax, 4
    mov ebx, 1
    mov ecx, NEWLINE
```

```
mov edx, 1

int 0x80

ret
```

Required Student Deliverables

Submit the .asm source file, screenshots of successful execution and invalid-input handling, and a brief explanation of implemented procedures.

Open-Ended Questions

1. What is the purpose of procedures in your application?
   - Procedures help organize my program by dividing it into smaller tasks. For example, get_user_info collects user information, determine_zodiac identifies the zodiac sign, and show_horoscope displays the predictions. Each procedure performs a specific function, making the program easier to understand and maintain. They also help avoid repeating the same code.
2. How do CALL and RET control program execution?
   - The CALL instruction is used to execute a specific procedure in the program. For example, CALL display_welcome displays the welcome screen before continuing to the next instruction. The RET instruction returns control to the instruction after the CALL. This allows the program to execute different procedures in an organized sequence.
3. Which conditional jump instructions did you use, and why?
   - I used conditional jump instructions such as JE, JNE, JL, JG, and JGE to control program decisions. The JE instruction checks menu choices, while JL and JG help validate user inputs. The JGE instruction is used in determine_zodiac to compare the birth day with the zodiac cutoff date. These instructions help the program select the correct action based on the user's input.
4. How does your program handle invalid user input?
   - My program checks user inputs before processing them. For example, validate_birthdate checks if the birth month and day are valid, while parse_uint checks whether the input contains a valid number. If the user enters an invalid date or menu option, the program displays an error message. It then allows the user to enter another value instead of immediately terminating.
5. What would be harder to maintain without procedures?

   - Without procedures, my program would be harder to read, debug, and modify because all the instructions would be combined. For example, changing the zodiac identification logic would be difficult if it were mixed with the menu and input instructions. Using procedures allows me to update one part of the program without affecting unrelated parts. This makes the program more organized and easier to improve.

Application-Specific Open-Ended Questions

Answer all questions for the application you developed. Write 3–5 complete sentences per question, refer to specific procedures or instructions from your own source code, and provide examples where requested.

Machine Problem 1 — Zodiac Horoscope and Fortune Teller

1. Explain how your determine_zodiac procedure identifies the correct sign from the birth month and day. Describe at least two conditional comparisons.

- The determine_zodiac procedure identifies the zodiac sign by comparing the user's birth month and day with the zodiac date ranges. It uses the CMP instruction to compare the birth day with the cutoff date. The JGE instruction checks if the day is greater than or equal to the cutoff. Based on the result, the program selects the correct zodiac sign.

2. How does your program handle zodiac date ranges that cross two months, such as Capricorn? Explain the branching logic.

- The program handles zodiac signs that cross two months by using the before_sign and after_sign arrays. For example, Capricorn covers December 22 to January 19. The program uses CMP and JGE instructions to determine which zodiac sign matches the entered date. This allows Capricorn to be identified correctly even though it crosses two months.

3. Why did you divide input collection, date validation, zodiac identification, and horoscope display into separate procedures?

- I divided the program into separate procedures to make the code organized and easier to understand. The get_user_info procedure collects user information, while validate_birthdate checks if the date is valid. The determine_zodiac procedure identifies the zodiac sign, and show_horoscope displays the prediction. This makes the program easier to debug and maintain.

4. Describe how your program rejects invalid dates such as February 30 and an invalid month such as 13.

- The program checks the entered month and day using conditional comparisons. If the user enters month 13, the program rejects it because the valid months are only 1 to 12. If the user enters February 30, the program checks the month_days array and detects that February only has 28 days. The program then displays an error message and asks the user to enter a valid date.

5. How does the program select a horoscope category after the user enters a menu choice? Identify the conditional jump instructions used.

- The program reads the user's menu choice and checks if it is between 1 and 6. It uses CMP, JL, and JG instructions to validate the selection. The show_horoscope procedure uses CMP and JE to determine which horoscope category to display. The program then shows the selected prediction or performs the corresponding menu action.

6. How did you implement the zodiac ASCII art? Explain how the correct design is selected and displayed.

- I implemented the zodiac ASCII art by storing different text designs for the 12 zodiac signs in the data section. Each design is connected to its corresponding zodiac sign through the arts array. The display_zodiac_art procedure uses the zodiac_index to select the correct design. It then displays the ASCII art along with the zodiac information.

7. What happens when a user enters an invalid menu option? Explain how control returns to the menu.

- When the user enters an invalid menu option, the program checks if the number is outside the valid range of 1 to 6. The JL or JG instruction directs the program to display an error message. After displaying the message, the JMP instruction returns to the menu loop. This allows the user to enter another choice without restarting the program.

8. If you added a new horoscope category, which procedures and conditional branches would you modify? Explain why.

- If I added a new horoscope category, I would modify the display_menu and show_horoscope procedures. I would add the new category to the menu and create a prediction for it in the data section. I would also add CMP and JE instructions to recognize the new selection. Lastly, I would update the menu validation and exit option so the new category works properly.

Machine Problem 2 — Dungeon Adventure and Survival Game

1. Explain how initialize_character assigns different health, attack, and potion values for Warrior, Mage, and Archer.

It assigns them by adding “je” to get the choices and whatever the user picks it will initialize their stats and potion values

2. Describe the procedures and conditional jumps used when the player chooses Attack, Defend, Use Potion, or Run.

Initialized a class called combat_attack, combat defend and combat potion and combat run these do a variety of things such as the attack option subtracting the enemies Health points depending on the class

3. How does your battle_enemy procedure determine whether the player or enemy has been defeated? Explain the comparisons.

It reads the players health then the enemy health then it will decide whether or not to proceed

4. Explain how you prevent health from exceeding the maximum and prevent potion use when the inventory is empty.

It just checks if it exceeds the maximum it will automatically set it back to their maximum health to the way it was originally using the cmp operation

5. How is the current dungeon room tracked, and what conditions allow the player to advance to the next room?

When the player selects Explore Dungeon, the program uses cmp and conditional jumps to check room_number and determine which room to display. And if they run they return back to the main menu but still have to face the enemy again but if they fail it will use conditional jumps to determine the player has been defeated.

6. What is the purpose of separating display_status, display_inventory, and battle_enemy into different procedures?

Because it requires them to have different procedures and it will help keep track and what to use and the reason why they are separated is because they are constantly updating like their health points and potion usage

7. How does your program preserve important register values across procedures and Linux system calls? Give one example.

I used pushad and popad For example, in the print_uint procedure, PUSHAD saves the register values before converting a number into digits, and POPAD restores them before returning. This prevents the procedure from accidentally changing register values that the rest of the program may still need.

8. If you introduced a fourth room with a stronger enemy, what program data, procedures, and conditions would need to change?

I introduce the stronger enemy with a higher health pool and attack but i compensate the player with potions upon the dragons defeat or add in a shop to upgrade the the damage of the player

Submission Requirement for Written Responses

Include a separate “Open-Ended Questions” section in your submission. Answer the five general questions and the eight questions for your assigned machine problem. Answers must reflect your actual program implementation; unsupported or generic explanations will not receive full credit.

Instructor Guidance

Assign Zodiac Horoscope first to focus on procedures, branching, and input validation. Assign Dungeon Adventure afterward for arithmetic, game state, repeated decisions, and interacting procedures.